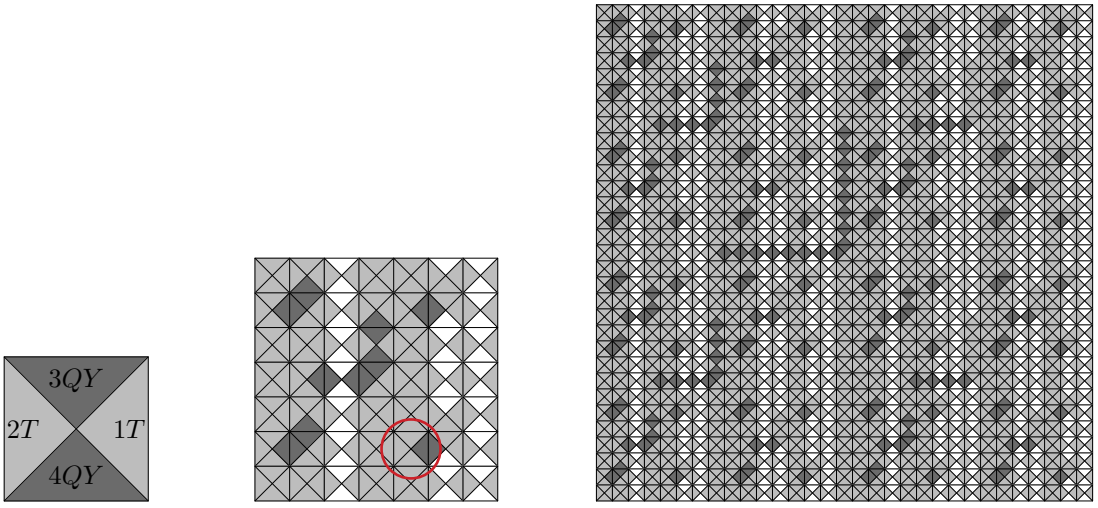
September 8, 2026 at 21:20

1. Introduction. A *tetrad* is a unit square cut into four triangles by its diagonals, each triangle carrying a symbol; two of them may be placed side by side only if the triangles that touch carry the same symbol, and they may not be rotated or reflected. Exercise 2.3.4.3–5 of Volume 1 exhibits 92 tetrad types that tile the plane, and proves that no tiling by them is periodic. Exercise 7.2.2.1–121 asks what Algorithm C can say about those 92:



βQTY A_3 A_5

(a) the tile βUS can appear in only $n+1$ cells of an $m \times n$ array, when $m, n \ge 4$, so it belongs to no infinite tiling;
(b) there is a unique $(2^k-1) \times (2^k-1)$ tiling whose middle tile is δRD, for every $k \ge 1$;
(c) there are exactly $(2, 2, 3, 57)$ of them whose middle tile is respectively δRU, δLD, δLU, δSU, for $k \ge 3$;
(d) only one tiling of the plane has each of those at the origin.

This program builds the 92 types from the specification on page 385 of Volume 1, turns the tiling question into an `XCC` problem—cells are the primary items, and the symbol on each shared edge is a color—and hands it to this repository's `XCC` engine. Every number above comes out, along with the structure that answer 121 gives for the solutions, the census of A_6, and the dragon sequence that answer 121 says lives on the edges of A_∞.

Two misprints turned up on the way, and both are already known. Part (c) of the exercise says $(2, 3, 3, 57)$; the errata of 7 January 2023 correct it to $(2, 2, 3, 57)$, which is what comes out here. And the 7×7 block printed in answer 2.3.4.3–5 of Volume 1 has γST in its sixth row, which is not one of the 92 types—the errata have said since 22 July 2005 that it should be γSJ, and §13 arrives at the same tile knowing only its neighbors. That cell is ringed in the picture above. Finding it is the best evidence I have that the 92 types were transcribed correctly.

```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strings"
    "time"
    cells "github.com/sjnam/dancing-cells"
)
```

```
⟨ Types 10 ⟩
⟨ Functions 4 ⟩
func main( ) {
    ⟨ Read the command line 2 ⟩
    ⟨ Do what the mode asks 3 ⟩
}
```

2. The heavy mode is *plane*, which searches a 31×31 board for each of the five middle tiles; the rest are seconds apiece.

```
⟨ Read the command line 2 ⟩ ≡
    mode := flag.String("mode", "all",
        "types, us, middle, corners, plane, census, or all")
    flag.Parse( )
    all := *mode ≡ "all"
```

This code is also defined in section 24.

This code is used in section 1.

3.

```
⟨ Do what the mode asks 3 ⟩ ≡
    if *mode ≡ "types" ∨ all {
        ⟨ Build the 92 types and check them 9 ⟩
    }
    if *mode ≡ "us" ∨ all {
        ⟨ See where βUS can go 20 ⟩
    }
    if *mode ≡ "middle" ∨ all {
        ⟨ Count the tilings with a given middle tile 23 ⟩
    }
    if *mode ≡ "corners" ∨ all {
        ⟨ Take the solutions apart into quadrants 28 ⟩
    }
    if *mode ≡ "plane" ∨ all {
        ⟨ Ask which tilings survive 33 ⟩
    }
    if *mode ≡ "census" ∨ all {
        ⟨ Census of A6, and the dragon sequence 35 ⟩
    }
```

This code is used in section 1.

4. The 92 types. Page 385 of Volume 1 gives 22 "basic codes," each a quadruple of symbols, and then names the 92 types as products of them. A type's four symbols are got by putting the codes together component by component and sorting each component into alphabetical order; the components are the top, bottom, left and right triangles, in that order. Thus βQTY, which Knuth works out as an example, is

$$(3,4,2,1)(Q,Q,\ ,\ )(\ ,\ ,T,T)(Y,Y,\ ,\ ) = (3QY, 4QY, 2T, 1T),$$

the tile drawn at the left of the picture above.

Greek letters and one repeated name would take this program out of `ASCII`, so α, β, γ and δ are written A, B, C and E here, and the empty code—the one Volume 1 calls B—is written O. Nothing else changes; *printName* puts the Greek back when a name is printed.

⟨ Functions 4 ⟩ ≡

```
var code = map[byte][4]string{
    'A': {"1","2","1","2"}, 'B': {"3","4","2","1"},
    'C': {"2","1","3","4"}, 'E': {"4","3","4","3"},
    'a': {"Q","D","P","R"}, 'b': {"","","L","P"},
    'c': {"U","Q","T","S"}, 'd': {"","","S","T"},
    'N': {"Y","","X",""}, 'J': {"D","U","","X"},
    'K': {"","Y","R","L"}, 'O': {"","","",""},
    'R': {"","","R","R"}, 'L': {"","","L","L"},
    'P': {"","","P","P"}, 'S': {"","","S","S"},
    'T': {"","","T","T"}, 'X': {"","","X","X"},
    'Y': {"Y","Y","",""}, 'U': {"U","U","",""},
    'D': {"D","D","",""}, 'Q': {"Q","Q","",""},
}
func quarters(name string) [4]string {
    var parts [4][]string
    for i := 0; i < len(name); i++ {
        c, ok := code[name[i]]
        if ¬ok {
            panic("no basic code " + string(name[i]))
        }
        for k := 0; k < 4; k++ {
            if c[k] ≠ "" {
                parts[k] = append(parts[k], c[k])
            }
        }
    }
    var q [4]string
    for k := 0; k < 4; k++ {
        sort.Strings(parts[k])
        q[k] = strings.Join(parts[k], "")
    }
    return q
}
```

This code is also defined in sections 5, 6, 7, 8, 14, 18, 19, 26, 32, 36.

This code is used in section 1.

5. Since the codes are put together component by component, they commute: the name says which codes a type uses, not in what order. That is worth knowing, because Volume 1 writes γPXB where this program writes $CXPO$, and the errata write δLJ for what is here EJL. They are the same tiles.

⟨ Functions 4 ⟩ +≡

```
func printName(name string) string {
    r := strings.NewReplacer("A", "\u03b1", "B", "\u03b2",
        "C", "\u03b3", "E", "\u03b4", "O", "B")
    return r.Replace(name)
}
```

6. The type list itself. Volume 1 writes it as four products, of 4, 21, 22 and 45 types; *product* expands one of them, taking a string of alternatives for each factor.

⟨ Functions 4 ⟩ +≡

```
func product(factors ... string) []string {
    out := []string{""}
    for _, f := range factors {
        var next []string
        for _, p := range out {
            for i := 0; i < len(f); i++ {
                next = append(next, p + string(f[i]))
            }
        }
        out = next
    }
    return out
}
```

7. ⟨ Functions 4 ⟩ +≡

```
func allTypes() []string {
    var ts []string
    add := func(gs ... []string) {
        for _, g := range gs {
            ts = append(ts, g...)
        }
    }
    add(product("A", "abcd"))
    add(product("B", "Y", "OUQ", "PT"), product("B", "OUDQ", "PST"),
        product("B", "K", "OUQ"))
    add(product("C", "XO", "LPST", "OQ"), product("C", "R", "OQ"),
        product("C", "J", "LPST"))
    add(product("E", "X", "LPST", "OQ"), product("E", "Y", "OUQ", "PT"),
        product("E", "N", "abcd"), product("E", "J", "LPST"),
        product("E", "K", "OUQ"), product("E", "RLPST", "OUDQ"))
    return ts
}
```

8. Everything below works with type numbers, so the list, the four symbols of each type, and the two tables that go between numbers and names are built once.

⟨ Functions 4 ⟩ +≡

```
var types = allTypes()
var quads = func() [][4]string {
    q := make([][4]string, len(types))
    for i, t := range types {
        q[i] = quarters(t)
    }
    return q
}()
var number = func() map[string]int {
    m := map[string]int{}
    for i, t := range types {
        m[t] = i
    }
    return m
}()
```

9. The first checks are that there are 92 of them, that no two are the same tile, and that βQTY comes out as Volume 1 says.

⟨ Build the 92 types and check them 9 ⟩ ≡

```
fmt.Printf("%d tetrad types built from the codes of exercise 2.3.4.3-5\n",
    len(types))
seen := map[[4]string]string{}
for i, q := range quads {
    if prev, dup := seen[q]; dup {
        fmt.Printf("  %s and %s are the same tile\n",
            printName(prev), printName(types[i]))
    }
    seen[q] = types[i]
}
fmt.Printf("  %d distinct tiles among them\n", len(seen))
fmt.Printf("  BQTY = %v, and Volume 1 says (3QY, 4QY, 2T, 1T)\n",
    quarters("BQTY"))
⟨ Check the block printed in answer 2.3.4.3–5 11 ⟩
```

This code is used in section 3.

10. Answer 2.3.4.3–5 of Volume 1 prints a whole 7×7 block of a tiling, 49 names, and that is the best test the transcription above can be given: if the codes, the composition rule or the order of the components were wrong, the block would not hold together. It nearly does. Every name but one is among the 92, and every edge but the four around that one name matches.

⟨ Types 10 ⟩ ≡

```
var printed = [7][7]string{
    {"Aa", "BKQ", "Ab", "BQP", "Aa", "BOK", "Ab"},
    {"CPJ", "ENa", "CRO", "EQK", "CLJ", "ENb", "CPO"},
    {"Ac", "BDS", "Ad", "BQTY", "Ac", "BOS", "Ad"},
    {"CPQ", "EPJ", "CPXO", "ENa", "CRQ", "ERO", "CRO"},
    {"Aa", "BUK", "Ab", "BDP", "Aa", "BOK", "Ab"},
    {"CTJ", "ENc", "CSO", "EDS", "CST", "ENd", "CTO"},
    {"Ac", "BQS", "Ad", "BDT", "Ac", "BOS", "Ad"},
}
```

This code is also defined in section 17.

This code is used in section 1.

11. ⟨ Check the block printed in answer 2.3.4.3–5 11 ⟩ ≡

```
var q [7][7][4]string
for i := 0; i < 7; i++ {
    for j := 0; j < 7; j++ {
        q[i][j] = quarters(printed[i][j])
        if _, ok := seen[q[i][j]]; ¬ok {
            fmt.Printf("␣␣the␣block's␣%s␣at␣row␣%d,␣column␣%d␣is␣not" +
                "␣one␣of␣the␣92\n", printName(printed[i][j]), i + 1, j + 1)
        }
    }
}
⟨ Count the mismatched edges of the block 12 ⟩
```

This code is used in section 9.

12. ⟨ Count the mismatched edges of the block 12 ⟩ ≡

```
bad := 0
for i := 0; i < 7; i++ {
    for j := 0; j < 7; j++ {
        if j + 1 < 7 ∧ q[i][j][3] ≠ q[i][j + 1][2] {
            bad++
        }
        if i + 1 < 7 ∧ q[i][j][1] ≠ q[i + 1][j][0] {
            bad++
        }
    }
}
fmt.Printf("␣␣the␣printed␣block:␣%d␣of␣its␣84␣internal␣edges␣do␣not␣match\n",
    bad)
⟨ Work out what the odd cell has to be 13 ⟩
```

This code is used in section 11.

13. Its four neighbors leave no choice, and the tile they ask for is *CJS*. Putting it in makes the block whole.

⟨ Work out what the odd cell has to be 13 ⟩ ≡

```
want := [4]string{q[4][4][1], q[6][4][0], q[5][3][3], q[5][5][2]}
fmt.Printf("␣␣its␣neighbors␣ask␣for␣%v,␣which␣is␣%s\n",
    want, printName(seen[want]))
printed[5][4] = seen[want]
bad = 0
for i := 0; i < 7; i++ {
    for j := 0; j < 7; j++ {
        q[i][j] = quarters(printed[i][j])
    }
}
for i := 0; i < 7; i++ {
    for j := 0; j < 7; j++ {
        if j + 1 < 7 ∧ q[i][j][3] ≠ q[i][j + 1][2] {
            bad++
        }
        if i + 1 < 7 ∧ q[i][j][1] ≠ q[i + 1][j][0] {
            bad++
        }
    }
}
fmt.Printf("␣␣with␣that␣tile␣in␣place:␣%d␣mismatched␣edges\n", bad)
```

This code is used in section 12.

14. The exact cover problem. A tiling of an $n \times n$ board is an `XCC` problem in the shape this repository's other pictures use: one primary item per cell, and one secondary item per interior edge, colored with the symbol the two tiles must agree on. Each option places one type in one cell and colors its up to four edges. A fifth secondary item carries the type number, so that a solution can be read straight back as a grid of numbers.

⟨ Functions 4 ⟩ +≡

```
func problem(rows, cols int, fix map[[2]int]int) string {
    var sb strings.Builder
    ⟨ Name the items 15 ⟩
    ⟨ Write one option for every type in every cell 16 ⟩
    return sb.String()
}
```

15. ⟨ Name the items 15 ⟩ ≡

```
for i := 0; i < rows; i++ {
    for j := 0; j < cols; j++ {
        fmt.Fprintf(&sb, "p%d.%d␣", i, j)
    }
}
sb.WriteString("|")
for i := 0; i < rows; i++ {
    for j := 0; j < cols; j++ {
        fmt.Fprintf(&sb, "␣n%d.%d", i, j)
    }
}
for i := 0; i < rows; i++ {
    for j := 0; j+1 < cols; j++ {
        fmt.Fprintf(&sb, "␣v%d.%d", i, j)
    }
}
for i := 0; i+1 < rows; i++ {
    for j := 0; j < cols; j++ {
        fmt.Fprintf(&sb, "␣h%d.%d", i, j)
    }
}
sb.WriteString("\n")
```

This code is used in section 14.

16. A cell named in *fix* gets the one option that pins it.

⟨ Write one option for every type in every cell 16 ⟩ ≡

```
for i := 0; i < rows; i++ {
    for j := 0; j < cols; j++ {
        pin, pinned := fix[[2]int{i, j}]
        for t := range types {
            if pinned ∧ t ≠ pin {
                continue
            }
            q := quads[t]
            fmt.Fprintf(&sb, "p%d.%d␣n%d.%d:t%d", i, j, i, j, t)
            if j > 0 {
                fmt.Fprintf(&sb, "␣v%d.%d:%s", i, j − 1, q[2])
            }
            if j + 1 < cols {
                fmt.Fprintf(&sb, "␣v%d.%d:%s", i, j, q[3])
            }
            if i > 0 {
                fmt.Fprintf(&sb, "␣h%d.%d:%s", i − 1, j, q[0])
            }
            if i + 1 < rows {
                fmt.Fprintf(&sb, "␣h%d.%d:%s", i, j, q[1])
            }
            sb.WriteString("\n")
        }
    }
}
```

This code is used in section 14.

17. ⟨ Types 10 ⟩ +≡

```
type grid [][]int
```

18. ⟨ Functions 4 ⟩ +≡

```
func solve(rows, cols int, fix map[[2]int]int, limit int) []grid {
    var out []grid
    for sol := range cells.NewXCC().Dance(
        strings.NewReader(problem(rows, cols, fix))).Solutions {
        g := make(grid, rows)
        for i := range g {
            g[i] = make([]int, cols)
        }
        for _, opt := range sol {
            var i, j, t int
            fmt.Sscanf(opt[1], "n%d.%d:t%d", &i, &j, &t)
            g[i][j] = t
        }
        out = append(out, g)
        if limit > 0 ∧ len(out) ≥ limit {
            break
        }
    }
    return out
}
```

19. Three small things a grid must do: compare itself with another, hand over one of its square blocks, and be pinned into a problem.

⟨ Functions 4 ⟩ +≡

```
func (a grid) same(b grid) bool {
    for i := range a {
        for j := range a[i] {
            if a[i][j] ≠ b[i][j] {
                return false
            }
        }
    }
    return true
}
func (a grid) block(r, c, h int) grid {
    b := make(grid, h)
    for i := range b {
        b[i] = append([]int(Λ), a[r + i][c:c + h]...)
    }
    return b
}
func (a grid) pin(fix map[[2]int]int, r, c int) {
    for i := range a {
        for j := range a[i] {
            fix[[2]int{r + i, c + j}] = a[i][j]
        }
    }
}
```

20. Where βUS can go. "Show that the tile called βUS can't be part of any infinite tiling. In fact, it can appear in only $n+1$ cells of an $m \times n$ array, when $m, n \geq 4$." Answer 121 gets there in two steps: there are no 2×2 solutions with βUS at the lower right, and no 3×4 solutions with it at the lower left, so it can appear only in the top row or at the left of the next-to-top row—which is $n+1$ cells.

⟨ See where βUS can go 20 ⟩ ≡

```
fmt.Println("Where␣beta-US␣can␣go")
us := number["BUS"]
⟨ Rule out the two corners answer 121 names 21 ⟩
for _, s := range [][2]int {{4, 4}, {4, 6}, {5, 5}, {6, 4} }{
    ⟨ Report the cells of an m × n array that βUS can reach 22 ⟩
}
```

This code is used in section 3.

21. "There are no 2×2 solutions with βUS at lower right; similarly, there are no 3×4 solutions with βUS at lower left." Those two facts are what force the $n+1$: a βUS anywhere but the top row and the left column has a 2×2 above and to its left, and one below the second row on the left edge has a 3×4.

⟨ Rule out the two corners answer 121 names 21 ⟩ ≡

```
for _, c := range []struct {
    rows, cols, i, j int
    where string
}{
    {2, 2, 1, 1, "lower␣right"},
    {3, 4, 2, 0, "lower␣left"},
}{
    got := len(solve(c.rows, c.cols,
        map[[2]int]int{{c.i, c.j}: us}, 1))
    fmt.Printf("␣␣%dx%d␣with␣it␣at␣the␣%s:␣%d␣tilings\n",
        c.rows, c.cols, c.where, got)
}
```

This code is used in section 20.

22. ⟨ Report the cells of an $m \times n$ array that βUS can reach 22 ⟩ ≡

```
m, n := s[0], s[1]
var ok [][2]int
for i := 0; i < m; i++ {
    for j := 0; j < n; j++ {
        if len(solve(m, n, map[[2]int]int{{i, j}: us}, 1)) > 0 {
            ok = append(ok, [2]int{i, j})
        }
    }
}
fmt.Printf("␣␣%dx%d:␣%d␣of␣the␣%d␣cells␣admit␣it,␣and␣n+1␣=␣%d\n",
    m, n, len(ok), m * n, n + 1)
fmt.Printf("␣␣␣␣they␣are␣%v\n", ok)
```

This code is used in section 20.

23. The tilings with a given middle tile. Part (b) asks for the $(2^k - 1) \times (2^k - 1)$ tilings whose middle tile is δRD, and part (c) for four more middles. One call apiece.

⟨ Count the tilings with a given middle tile 23 ⟩ ≡

```
fmt.Println("Tilings of a (2^k-1)x(2^k-1) board with a given middle tile")
for _, name := range []string { "ERD", "ERU", "ELD", "ELU", "ESU" }{
    fmt.Printf("  %s", printName(name))
    for k := 1; k ≤ *deep; k++ {
        n := 1 ≪ k − 1
        t := time.Now()
        got := len(solve(n, n, map[[2]int]int{{n/2, n/2}: number[name]}, 0))
        fmt.Printf("  %dx%d: %-3d(%s)", n, n, got,
            time.Since(t).Round(time.Millisecond))
    }
    fmt.Println()
}
⟨ Try all 92 in the middle of a 7 × 7 board 25 ⟩
```

This code is used in section 3.

24. ⟨ Read the command line 2 ⟩ +≡

```
deep := flag.Int("k", 4, "how far to push the (2^k-1)x(2^k-1) boards")
```

25. One tile in 92 admits no 7×7 tiling around it at all, and it is βUS—which is what part (a) says it must be.

⟨ Try all 92 in the middle of a 7 × 7 board 25 ⟩ ≡

```
none, total := []string{}, 0
for t := range types {
    got := len(solve(7, 7, map[[2]int]int{{3, 3}: t}, 0))
    total += got
    if got ≡ 0 {
        none = append(none, printName(types[t]))
    }
}
fmt.Printf("  all 92 in the middle of a 7x7 board: %d tilings in all," +
    " and %v admits none\n", total, none)
```

This code is used in section 23.

26. The quadrants. Answer 121 does not merely count these tilings, it says what they look like. Let A_k, B_k, C_k, D_k be the $(2^k-1)\times(2^k-1)$ tilings that have αa, αb, αc, αd when $k=1$, and otherwise $\delta N a$, $\delta N b$, $\delta N c$, $\delta N d$ in the middle with A_{k-1}, B_{k-1}, C_{k-1}, D_{k-1} at the corners in reading order. The cross between the corners is then forced—*quadrants* checks that too, by asking for every way to fill it.

```
⟨ Functions 4 ⟩ +≡
    var cache = map[int][4]grid{ }
    func quadrants(k int) [4]grid {
        if q, ok := cache[k]; ok {
            return q
        }
        var out [4]grid
        if k ≡ 1 {
            for i, name := range []string { "Aa", "Ab", "Ac", "Ad" }{
                out[i] = grid{{number[name]}}
            }
        } else {
            ⟨ Grow the four blocks from the four smaller ones 27 ⟩
        }
        cache[k] = out
        return out
    }
```

27. ⟨ Grow the four blocks from the four smaller ones 27 ⟩ ≡

```
    prev := quadrants(k − 1)
    n, h := 1 ≪ k − 1, 1 ≪ (k − 1) − 1
    for i, mid := range []string { "ENa", "ENb", "ENc", "ENd" }{
        fix := map[[2]int]int{ }
        prev[0].pin(fix, 0, 0)
        prev[1].pin(fix, 0, h + 1)
        prev[2].pin(fix, h + 1, 0)
        prev[3].pin(fix, h + 1, h + 1)
        fix[[2]int{h, h}] = number[mid]
        sols := solve(n, n, fix, 0)
        if len(sols) ≠ 1 {
            fmt.Printf("␣␣the␣cross␣of␣level␣%d,␣middle␣%s,␣can␣be␣filled␣in" +
                "␣%d␣ways\n", k, printName(mid), len(sols))
        }
        out[i] = sols[0]
    }
```

This code is used in section 26.

28. Now the claims. Answer 121(b) says the unique tiling with δRD in the middle has D_{k-1}, C_{k-1}, B_{k-1}, A_{k-1} at its corners; the errata of 8 January 2023 rewrite (c) to say that δRU gains a solution with corners C, D, A, B, that δLD gains one with corners B, A, D, C, that both of those work for δLU and δSU as well, and that δSU has 54 more besides.

```
⟨ Take the solutions apart into quadrants 28 ⟩ ≡
    fmt.Println("What␣the␣solutions␣look␣like")
    k := *deep
    n, h := 1 ≪ k − 1, 1 ≪ (k − 1) − 1
    prev := quadrants(k − 1)
    letters := []string{"A", "B", "C", "D"}
    for _, name := range []string { "ERD", "ERU", "ELD", "ELU", "ESU" }{
        sols := solve(n, n, map[[2]int]int{{h, h}: number[name]}, 0)
        ⟨ Tally the corners of each solution 29 ⟩
    }
    ⟨ Look at the 54 solutions that are not made of quadrants 30 ⟩
```

This code is used in section 3.

29.

```
⟨ Tally the corners of each solution 29 ⟩ ≡
    tally := map[string]int{}
    for _, s := range sols {
        corners := ""
        for _, c := range [][2]int {{0, 0}, {0, h + 1}, {h + 1, 0}, {h + 1, h + 1} }{
            which := "?"
            for i, p := range prev {
                if s.block(c[0], c[1], h).same(p) {
                    which = letters[i]
                }
            }
            corners += which
        }
        tally[corners]++
    }
    keys := []string{}
    for c := range tally {
        keys = append(keys, c)
    }
    sort.Strings(keys)
    fmt.Printf("␣␣%s␣on␣%dx%d:␣%d␣tilings;", printName(name), n, n, len(sols))
    for _, c := range keys {
        fmt.Printf("␣␣corners␣%s:␣%d", c, tally[c])
    }
    fmt.Println()
```

This code is used in section 28.

30. "And δSU also has 54 additional solutions, with C_{k-2} in the upper left corner. They use $\delta\{\{L,P,S,T\}\{J,U\},RU\}$ in the middle of row 2^{k-2}, and independently $\delta\{L,K,P,R,S,T\}U$ in the middle of row $3\cdot 2^{k-2}$." Nine tiles in one row and six in the other, chosen independently, is $9\times 6=54$, and that is exactly what happens: all 54 pairs occur, once each.

```
⟨ Look at the 54 solutions that are not made of quadrants 30 ⟩ ≡
    sols := solve(n, n, map[[2]int]int{{h, h}: number["ESU"]}, 0)
    var extra []grid
    for _, s := range sols {
        ordinary := false
        for _, p := range prev {
            if s.block(0, 0, h).same(p) {
                ordinary = true
            }
        }
        if ¬ordinary {
            extra = append(extra, s)
        }
    }
    ⟨ Report the shape of those 54 31 ⟩
```

This code is used in section 28.

31.

```
⟨ Report the shape of those 54 31 ⟩ ≡
    c := quadrants(k − 2)[2]
    hh, off := 1 ≪ (k − 2) − 1, 0
    for _, s := range extra {
        if ¬s.block(0, 0, hh).same(c) {
            off ++
        }
    }
    fmt.Printf("␣␣and␣%d␣more,␣%d␣of␣which␣fail␣to␣have␣C_%d␣in␣the␣upper␣left\n",
        len(extra), off, k − 2)
    r1, r2, mid := 1 ≪ (k − 2) − 1, 3 ∗ (1 ≪ (k − 2)) − 1, h
    one, two := map[int]bool{}, map[int]bool{}
    pairs := map[[2]int]bool{}
    for _, s := range extra {
        one[s[r1][mid]] = true
        two[s[r2][mid]] = true
        pairs[[2]int{s[r1][mid], s[r2][mid]}] = true
    }
    fmt.Printf("␣␣␣␣row␣%d␣holds␣%d␣tiles,␣row␣%d␣holds␣%d,␣and␣%d␣of␣the␣%d" +
        "␣pairs␣occur\n", r1 + 1, len(one), r2 + 1, len(two), len(pairs),
        len(one) ∗ len(two))
    fmt.Printf("␣␣␣␣row␣%d:␣%s\n", r1 + 1, listing(one))
    fmt.Printf("␣␣␣␣row␣%d:␣%s\n", r2 + 1, listing(two))
```

This code is used in section 30.

32. ⟨ Functions 4 ⟩ +≡

```
func listing(m map[int]bool) string {
    var v []string
    for t := range m {
        v = append(v, printName(types[t]))
    }
    sort.Strings(v)
    return strings.Join(v, "␣")
}
```

33. Which tilings survive. Part (d) asks how many tilings of the whole plane have each of those tiles at the origin, and the answer is "only one of each." A tiling of the plane restricts, for every k, to a $(2^k - 1) \times (2^k - 1)$ tiling with that tile in the middle; so a solution that is not the middle of some larger solution is already dead. One level of extension kills all but one for four of the five tiles. For δSU it takes two.

⟨ Ask which tilings survive 33 ⟩ ≡

```
fmt.Println("Which␣tilings␣extend␣to␣larger␣boards")
for _, name := range []string { "ERD", "ERU", "ELD", "ELU", "ESU" }{
    base := solve(7, 7, map[[2]int]int{{3, 3}: number[name]}, 0)
    for _, k := range []int { 4, 5 }{
        ⟨ Count the 7 × 7 tilings that are middles of larger ones 34 ⟩
    }
}
```

This code is used in section 3.

34. ⟨ Count the 7 × 7 tilings that are middles of larger ones 34 ⟩ ≡

```
n, h := 1 ≪ k − 1, 1 ≪ (k − 1) − 1
t := time.Now( )
up := solve(n, n, map[[2]int]int{{h, h}: number[name]}, 0)
live := map[int]bool{ }
for _, s := range up {
    c := s.block(h − 3, h − 3, 7)
    for i, b := range base {
        if c.same(b) {
            live[i] = true
        }
    }
}
fmt.Printf("␣␣%s:␣%d␣of␣the␣%d␣tilings␣of␣7x7␣are␣the␣middle␣of␣one␣of␣the" +
    "␣%d␣tilings␣of␣%dx%d␣(%s)\n", printName(name), len(live), len(base),
    len(up), n, n, time.Since(t).Round(time.Second))
```

This code is used in section 33.

35. The census, and the dragon. Answer 121 closes with two remarks in brackets. The first: "Each of the other 86 types occurs in A_6, hence in every sufficiently large tiling." The other six are βUS and the five tiles of parts (b) and (c), and A_6 is 63×63, so this is a census.

⟨ Census of A_6, and the dragon sequence 35 ⟩ ≡

```
fmt.Println("The census of A_6")
a := quadrants(6)[0]
used := map[int]bool{}
for _, row := range a {
    for _, t := range row {
        used[t] = true
    }
}
var absent []string
for t := range types {
    if ¬used[t] {
        absent = append(absent, printName(types[t]))
    }
}
sort.Strings(absent)
fmt.Printf("  A_6 is %dx%d and uses %d of the 92 types\n",
    len(a), len(a), len(used))
fmt.Printf("  missing: %s\n", strings.Join(absent, " "))
⟨ Look for the dragon sequence on the edges 37 ⟩
```

This code is used in section 3.

36. The second remark: "the 'dragon sequence' (see answer 4.5.3–41) arises in the colors at the edges of A_∞, B_∞, C_∞, D_∞." That sequence is $d_0 = 1$, $d_{2n} = d_n$, $d_{4n+1} = 0$, $d_{4n+3} = 1$, and it does arise, as plainly as one could wish. Along the top edge of A_k the triangles read $1Q$, $3Q$, 1, $3Q$, $1Q$, 3, 1, $3Q$, ..., and down its left edge $1P$, $3P$, $1T$, $3P$, $1P$, $3T$, $1T$, $3P$, ...; in both the nth of them carries a Q or a P exactly when $d_n = 0$. That is the whole edge, every place of it. In B_k, C_k and D_k it is right everywhere but the middle of the edge, which is where the next level's cross attaches.

⟨ Functions 4 ⟩ +≡

```
func dragon(n int) []int {
    d := make([]int, n)
    d[0] = 1
    for i := 1; i < n; i++ {
        switch {
        case i % 2 ≡ 0:
            d[i] = d[i/2]
        case i % 4 ≡ 1:
            d[i] = 0
        default:
            d[i] = 1
        }
    }
    return d
}
```

37. ⟨ Look for the dragon sequence on the edges 37 ⟩ ≡

```
letters := []string{"A", "B", "C", "D"}
for k := 3; k ≤ 6; k++ {
    q := quadrants(k)
    n := len(q[0])
    d := dragon(n + 2)
    for w, g := range q {
        top, left := 0, 0
        for j := 0; j < n; j++ {
            if strings.ContainsAny(quads[g[0][j]][0], "QP") ≡ (d[j + 1] ≡ 0) {
                top++
            }
            if strings.ContainsAny(quads[g[j][0]][2], "QP") ≡ (d[j + 1] ≡ 0) {
                left++
            }
        }
        fmt.Printf("␣␣%s_%d:␣the␣dragon␣gets␣%d␣of␣%d␣places␣along␣the␣top" +
            "␣edge␣right,␣and␣%d␣of␣%d␣down␣the␣left\n",
            letters[w], k, top, n, left, n)
    }
}
```

This code is used in section 35.

38. Index.

Wang's tetrads, and the tilings they force